



Predavanje 3

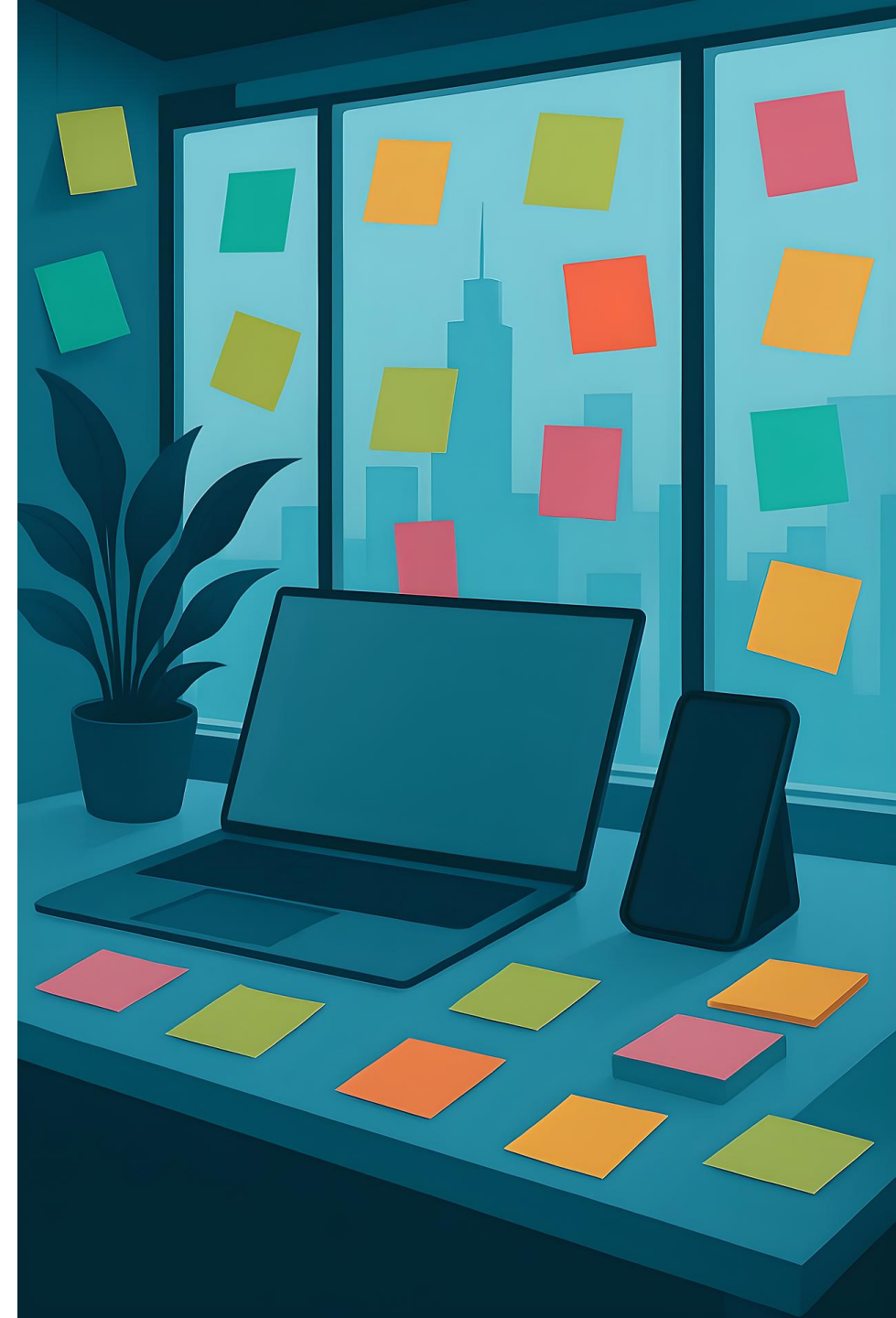
Programiranje za UNIX

Doc. dr. sc. Maja Braović

16. listopada 2025.

Sadržaj

- Uvod
- Korisne UNIX naredbe
- X Windows sustav (X11)
- SSH protokol
- Prevođenje i povezivanje programa
 - GNU C prevodioc
 - Kolekcije funkcija (libovi)
 - Rutina **make**
 - Struktura Makefile datoteke
- Literatura



Korisne UNIX naredbe

- Ukoliko se pomoću komandne linije spajate na neko udaljeno računalo, sljedeće naredbe će vam biti korisne:
 - pwd (print working directory)
 - cd (change directory)
 - ls (list)
 - cp (copy)
 - mv (move)
 - rm (remove)
 - mkdir (make directory)
 - rmdir (remove directory)

Naredba **pwd**

- Naredba **pwd** (**p**rint **w**orking **d**irectory) koristi se za prikaz trenutnog direktorija u kojem se nalazite.
- Primjer:

>> pwd	// naš upit
>> /home/username	// ispis
>> cd ..	// promjena direktorija
>> pwd	// naš upit
>> /home	// ispis

Naredba **cd**

- Naredba **cd** (**c**hange **d**irectory) koristi se za promjenu trenutnog direktorija.

- Primjer:

>> pwd	// naš upit
>> /darko/unix	// ispis
>> cd ..	// promjena direktorija
>> pwd	// naš upit
>> /darko	// ispis

Naredba **ls**

- Naredba **ls** (**list**) koristi se za ispis sadržaja tekućeg direktorija.
- Dodatne opcije:
 - **ls -a** (**list all**) – prikazuje i skrivene datoteke
 - **ls -l** (**list long**) – prikazuje detaljne informacije o sadržaju direktorija (npr. veličina datoteke, vlasnik datoteke, datum posljednje modifikacije datoteke)

Naredba **cp**

- Naredba **cp** (**copy**) koristi se za kopiranje datoteke sa jednog mjesta na neko drugo mjesto.
- Sintaksa: **cp <izvor> <odredište>**
- Primjer: **cp vjezba1.txt vjezba2.txt**
 - Gornja naredba kopira sadržaj datoteke vjezba1.txt u datoteku vjezba2.txt, pa će sada obje datoteke imati jednak sadržaj.
- Primjer: **cp vjezba1.txt labovi**
 - Gornja naredba kopira datoteku vjezba1.txt u direktorij labovi.
- Primjer: **cp vjezba1.txt labovi/moja_vjezba.txt**
 - Gornja naredba kopira datoteku vjezba1.txt u direktorij labovi pod novim nazivom (moja_vjezba.txt).

Naredba **mv**

- Naredba **mv** (**move**) koristi se za premještanje datoteke sa jednog mjesta na neko drugo mjesto.
- Sintaksa: **mv <izvor> <odredište>**
- Primjer: **mv vjezba1.txt vjezba2.txt**
 - Gornja naredba će promijeniti naziv datoteke vjezba1.txt u vjezba2.txt.
- Primjer: **mv vjezba1.txt labovi**
 - Gornja naredba premješta datoteku vjezba1.txt u direktorij labovi.
- Primjer: **mv vjezba1.txt labovi/moja_vjezba.txt**
 - Gornja naredba premješta datoteku vjezba1.txt u direktorij labovi pod novim nazivom (moja_vjezba.txt).

Naredba **rm**

- Naredba **rm** (**remove**) koristi se za brisanje datoteka.
- Sintaksa: **rm <ime_datoteke>**
- Primjer: **rm vjezba1.txt**
 - Gornja naredba će izbrisati datoteku vjezba1.txt.
- Primjer: **rm labovi/vjezba1.txt**
 - Gornja naredba će izbrisati datoteku vjezba1.txt koja se nalazi u direktoriju labovi.
- Primjer: **rm ***
 - Gornja naredba briše sve datoteke u direktoriju koje ne počinju sa točkom.
- Primjer: **rm -i ***
 - Gornja naredba briše sve datoteke u direktoriju koje ne počinju sa točkom, ali će vas pitati za potvrdu prije brisanja svake datoteke.

Naredba **mkdir**

- Naredba **mkdir** (**m**ake **d**irectory) koristi se za stvaranje novog direktorija.

>> pwd → /home/igor

>> ls → predavanje1.pdf

>> **mkdir** vjezba1 // stvaramo direktorij vjezba1

>> ls → predavanje1.pdf **vjezba1**

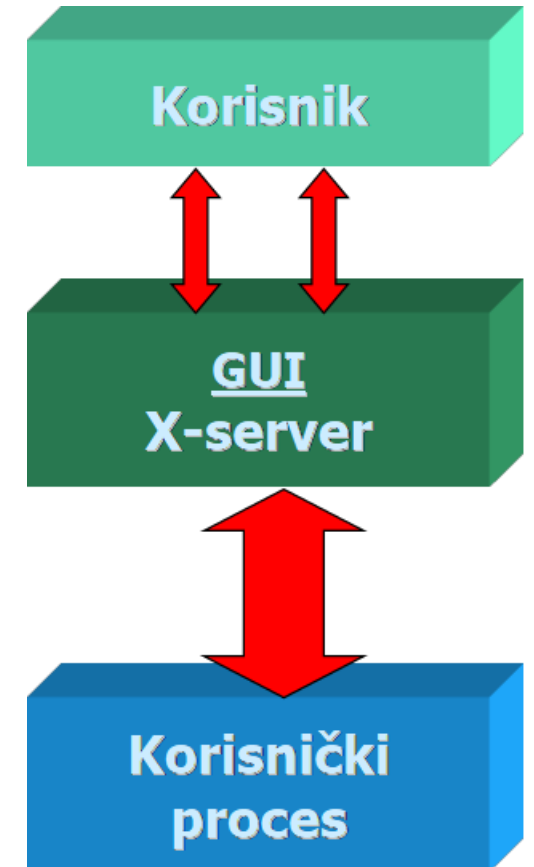
Naredba **rmdir**

- Naredba **rmdir** (**re**move **di**rectory) koristi se za brisanje postojećeg direktorija koji mora biti prazan. Ukoliko u direktoriju postoje neke datoteke, moramo ih obrisati prije brisanja direktorija.

```
>> pwd                →      /home/igor
>> ls                 →      predavanje1.pdf  vjezba1
>> rmdir vjezba1      // brišemo direktorij vjezba1
>> ls                 →      predavanje1.pdf
```

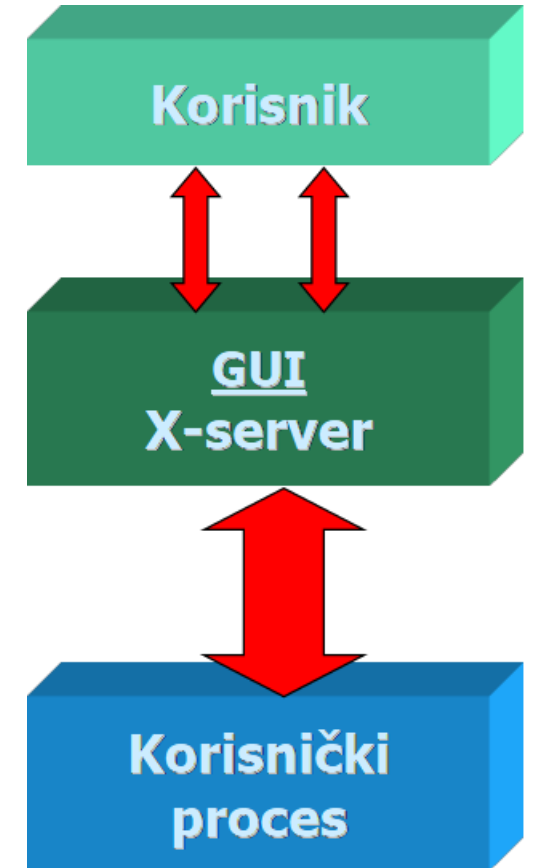
X Windows sustav

- X Windows sustav (X11, X) je grafički sustav koji se koristi kod UNIX i UNIX-sličnih operacijskih sustava za omogućavanje prikaza grafičkog sučelja.
- Nastao je 1984. godine na MIT-u, a od 1987. godine koristi se X Windows verzija 11, tj. X11.
- Prilikom korištenja X11 sustava proces i sučelje procesa prema korisniku ne moraju nužno biti na istom računalu.
- Arhitektura X11 sustava temelji se na modelu klijent-poslužitelj:
 - X server upravlja zaslonom i uređajima za unos podataka (tipkovnica, miš),
 - aplikacije (tj. klijenti) komuniciraju s X serverom kako bi prikazale svoj grafički sadržaj,
 - komunikacija između X servera i klijenata odvija se pomoću definiranog protokola.

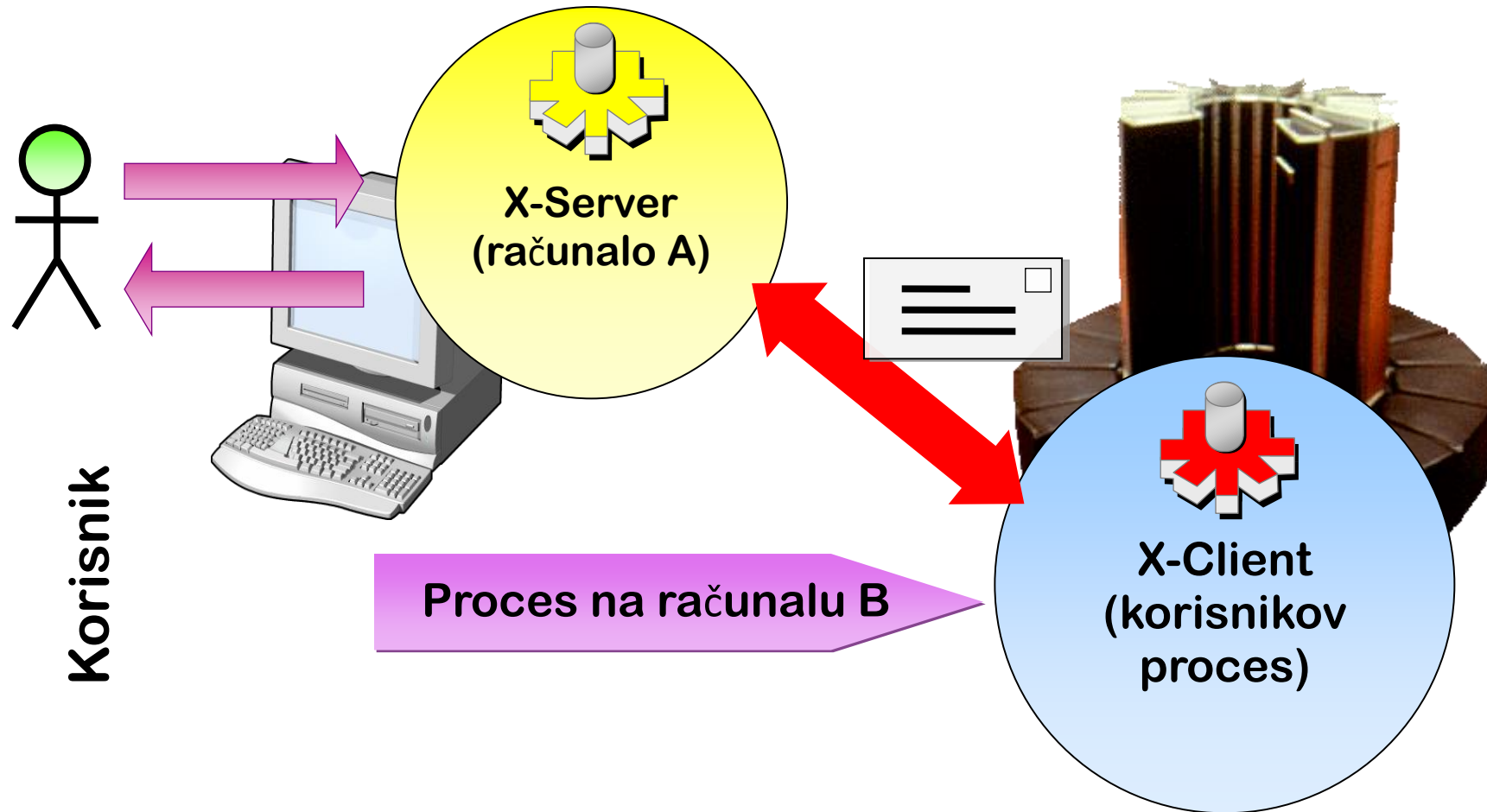


X Windows sustav

- U X Windows sustavu, X server se nalazi na korisničkom računalu, odnosno na računalu koje ima grafički zaslon i ulazne uređaje (tipkovnicu, miš).
- Aplikacije, koje se nazivaju klijenti, mogu biti pokrenute lokalno ili na udaljenim računalima.
- Klijenti šalju grafičke zahtjeve X serveru, a on ih prikazuje te također proslijeđuje korisničke ulazne događaje natrag klijentima.
- Ovo je dio mrežne transparentnosti sustava – aplikacije mogu biti na udaljenim računalima, dok je X server uvijek na korisničkoj strani gdje je fizički prikaz (zaslon i ulazni uređaji).



X Windows sustav

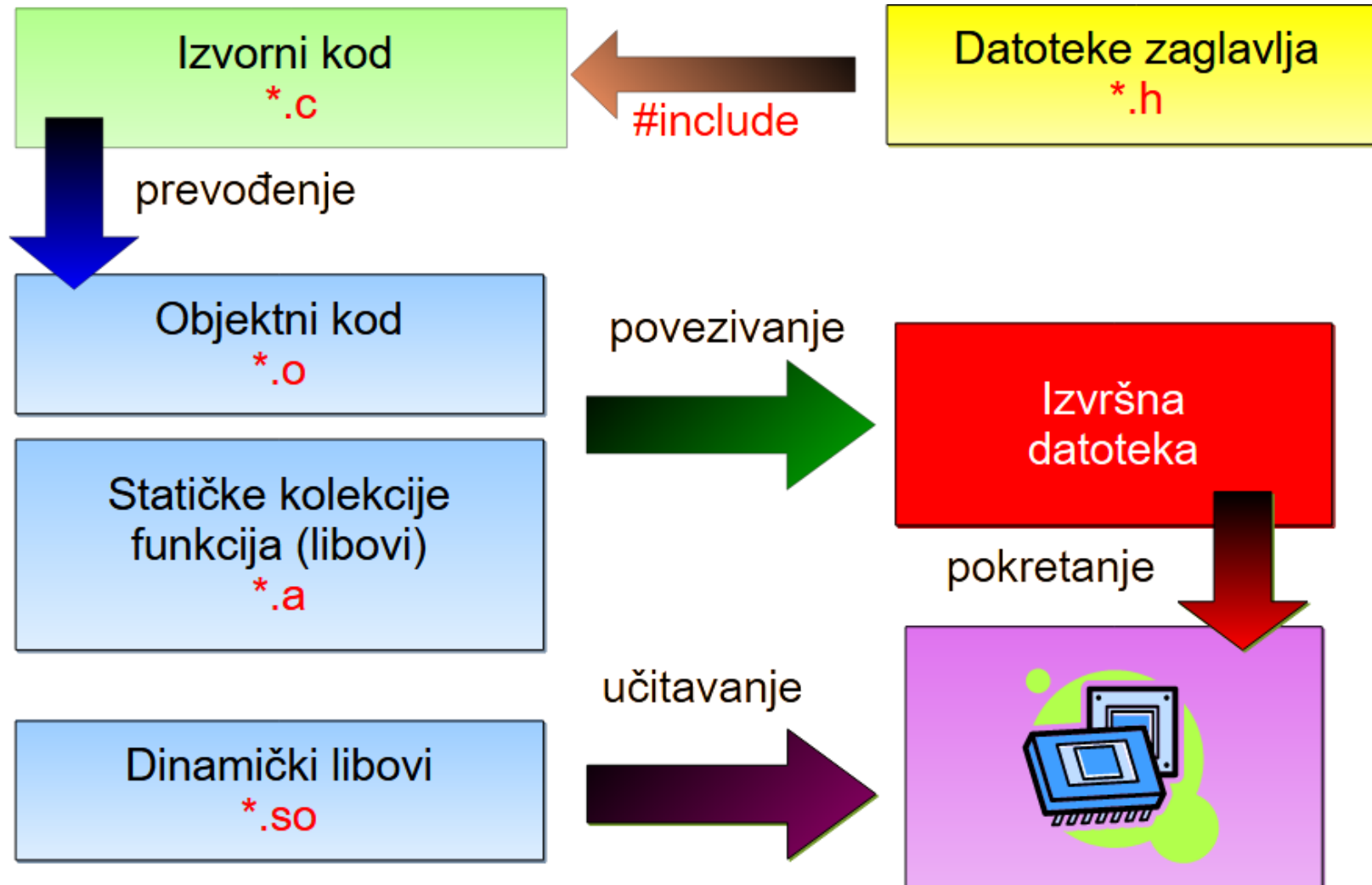


■ **ssh** – spajanje na udaljeno računalo

SSH protokol

- SSH (**S**ecure **S**Hell) je protokol za sigurno udaljeno povezivanje i upravljanje računalima preko mreže.
- Kreirao ga je 1995. godine Tatu Ylönen sa Tehničkog sveučilišta u Helsinkiju u Finskoj.
- Kreiran je kao zamjena za nesigurne protokole koji su se do tada koristili, i koji su koristili samo lozinke slane otvorenim tekstom za autentikaciju.
- SSH uvodi enkripcijske mehanizme koji skrivaju sadržaj podataka koji se šalju putem mreže, te omogućava provjeru autentičnosti korisnika pomoću kriptografskih ključeva.
- Standardni SSH protokol je 22, no može se promijeniti (npr. za zaštitu od automatiziranih napada).
- Za spajanje na server putem SSH protokola na Windows računalima može se koristiti **PuTTY**.
- Primjer: [ssh korisničko_ime@ime_servera.domena.hr](#)

Prevođenje i povezivanje programa

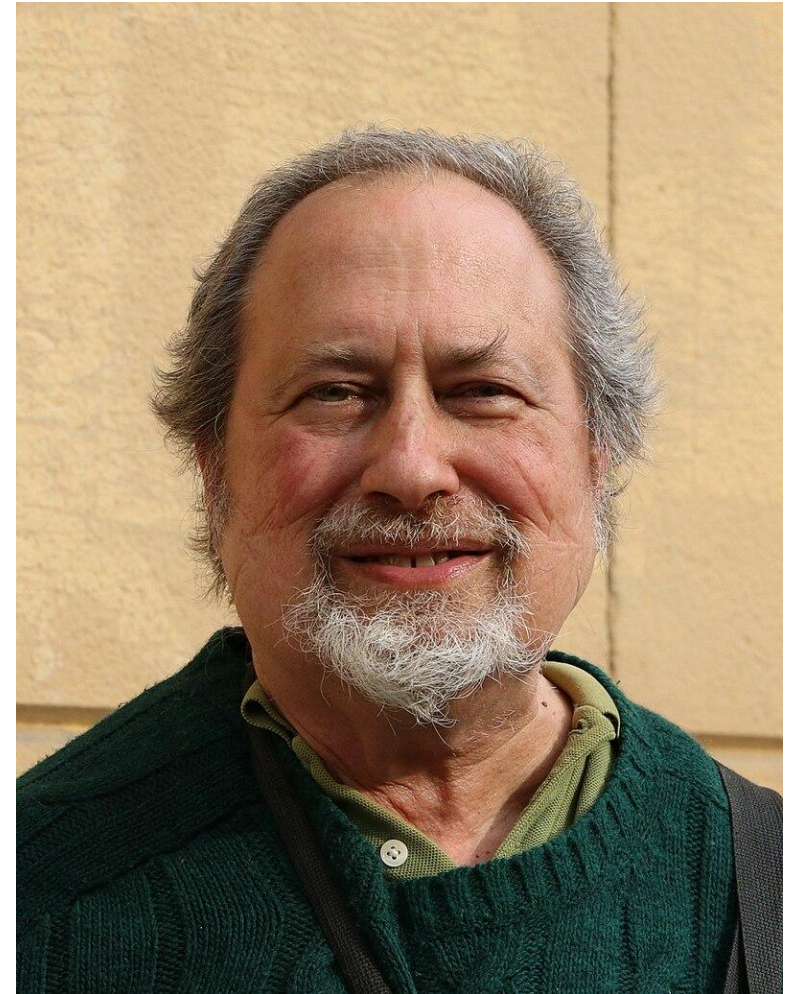


Prevođenje i povezivanje programa

- Prevođenje (eng. **compiling**) programa je proces prevođenja programskog koda u izvršnu formu.
- Povezivanje (eng. **linking**) programa je spajanje različitih dijelova tog koda u jednu cjelinu spremnu za izvođenje.
- Prevođenje i povezivanje su dva odvojena procesa.
- U operacijskim sustavima sličnim UNIX-u, za prevođenje i povezivanje programa koriste se alati nastali u okviru GNU projekta, **GCC** (GNU Compiler Collection) kompajler i GNU linker **ld**.
- Budući da su se „linkeri” originalno zvali „loaderi”, ime **ld** je skraćenica za **load**, ali može biti i skraćenica za „link editor”.
- Danas se pomoću naredbe gcc može obaviti i kompajliranje i linkanje, odnosno i prevođenje i povezivanje programa.

GNU projekt

- Cilj GNU projekta je da se razvije slobodan operacijski sustav i softver koji korisnicima daje 4 temeljna prava [2]:
 1. pravo pokretanja programa
 2. pravo proučavanja i modifikacije programskog koda programa
 3. pravo dijeljenja programa
 4. pravo dijeljenja modificiranih verzija programa.
- GNU projekt službeno je započeo 1985. godine **Richard M. Stallman** (prikazan na slici desno).
- GNU je skraćenica za „GNU's not UNIX”.



Slika preuzeta iz [1].

GNU C prevodioc (GCC)

- Prevođenje (eng. **compiling**) programa je proces prevođenja programskog koda u izvršnu formu.
- GNU C prevodilac je dio **GNU Compiler Collection (GCC)**, koji je skup kompajlera iz GNU projekta.
- Izvorno je bio poznat kao "GNU C Compiler" jer je inicijalno podržavao samo programski jezik C, ali je kasnije proširen da podržava i mnoge druge jezike poput C++, Fortrana, Ade, Go i dr.
- GCC je slobodan softver i široko korišten za prevođenje C programa, naročito na operacijskim sustavima sličnim UNIX-u.
- Sintaksa: **gcc [-opcija] ulazne_datoteke**
- Ulazne datoteke: ***.c, *.o, *.a, ...**

GNU C prevodioc (GCC)

- Osnovne opcije **gcc** naredbe:
 - **-c** – samo prevođenje (rezultat je objektna datoteka)
 - **-o** *ime* – ime izlazne datoteke
 - **-O***level* – razina optimizacije
 - **-g** – uključi informaciju za debugger
 - **-I***dir* – uključi *dir* u listu direktorija sa datotekama zaglavlja
 - **-L***dir* – uključi *dir* u listu direktorija sa kolekcijama funkcija (libovima)
 - **-Wall** – prikaži sva upozorenja
 - **-Werror** – upozorenja tretiraj kao greške
 - **-v** – uključuje dodatna objašnjenja koraka kompajliranja

GCC: opcija -c

- Uključivanjem ove opcije naredba **gcc** obavlja samo prevođenje (kompajliranje) programskog koda, a rezultat koji se dobije je objektna datoteka (ne izvršna datoteka).
- Primjer: **gcc -c primjer.c**
- Rezultat: objektna datoteka **primjer.o** koja sadrži kompajlirani kod spreman za povezivanje (linkanje) s drugim objektima ili bibliotekama u kasnijem koraku.
- Ova opcija omogućava programerima da:
 - mijenjaju i kompajliraju samo jedan dio koda, odnosno štede na vremenu i omogućavaju modularni razvoj
 - koriste različite verzije datoteka i biblioteka

GCC: opcija -o *ime*

- Uključivanjem ove opcije naredba **gcc** omogućava specificiranje imena izlazne datoteke koju će kompajler generirati.
- U slučaju da se opcija -o ne koristi, gcc kompajler će generirati izvršnu datoteku a.out.
- Primjer: **gcc -o primjer primjer.c** (pokreće se sa ./primjer)
- Primjer: **gcc primjer.c -o primjer** (pokreće se sa ./primjer)
- Primjer: **gcc primjer.c** (pokreće se sa ./a.out)

GCC: opcija **-O**level

- Uključivanjem ove opcije naredbom **gcc** postavlja se razina optimizacije kojom se određuje stupanj i vrste optimizacija koje će kompajler primijeniti tijekom generiranja izvršnog koda.
- Optimizacije omogućava programerima da odrede što im je važnije: brže kompajliranje ili brže izvođenje programa.
- Najčešće korištene optimizacijske razine su:
 - **-O0**: nema optimizacije. Kompajliranje je najbrže, no generirani kod je veći i sporiji.
 - **-O1**: optimizacija razine 1.
 - **-O2**: optimizacija razine 2.
 - **-O3**: najviša razina optimizacije koja uključuje složene tehnike za poboljšanje brzine izvođenja, često uz povećanje veličine izvršne datoteke.
 - **-Os**: optimizacija kojom se smanjuje veličina izvršne datoteke.

GCC: opcija -g

- Uključivanjem ove opcije naredba **gcc** omogućava jednostavnije praćenje i pronalaženje pogrešaka u programskom kodu, budući da se prilikom izrade izvršne datoteke generiraju i dodatni podaci za alate za debuggiranje (npr. za GNU Debugger, **gdb**).
- Ova opcija se često koristi zajedno s -O0 (bez optimizacije) kako bi se izbjegle poteškoće kod praćenja koda zbog promjena nastalih tijekom optimizacije.
- Primjer: **gcc -g primjer.c -o primjer**
gdb ./primjer

GCC: opcija **-ldir**

- Uključivanjem ove opcije naredba **gcc** omogućava povezivanje (linkanje) s određenom bibliotekom.
- Primjer: **gcc primjer.c -lm**
- Gornja naredba kompajlira primjer.c i povezuje ga s matematičkom bibliotekom koja omogućava korištenje funkcija poput sqrt, sin i cos.
- Dodatni primjeri:
 - **-lpthread** (POSIX niti)
 - **-lcrypto** (OpenSSL kriptografija)

GCC: opcija -Wall

- Uključivanjem ove opcije naredba **gcc** omogućava ispis većine standardnih upozorenja tijekom kompajliranja programa.
- Ime naredbe je skraćenica za „**W**arning **all**”.
- Primjer: **gcc -Wall primjer.c -o primjer**
- Gornja naredba kompajlira primjer.c i uključuje upozorenja koja pomažu u otkrivanju potencijalnih problema u izvornom kodu.

GCC: opcija -Werror

- Uključivanjem ove opcije naredba **gcc** omogućava tretiranje svih upozorenja kao greški prilikom kompajliranja koda.
- Ime naredbe je skraćenica za „**W**arning **error**”.
- Primjer: **gcc -Werror primjer.c -o primjer**
- Gornja naredba kompajlira primjer.c, a ukoliko se pojavi bilo kakvo upozorenje, kompajliranje se završava.

GCC: opcija **-v**

- Uključivanjem ove opcije naredba **gcc** omogućava detaljno prikazivanje svih koraka i komandi koje GCC koristi tijekom procesa kompajliranja i povezivanja programa.
- Ime naredbe je skraćenica za „**verbose**” (hrv. opširan).
- Primjer: **gcc -v primjer.c**
- Korištenjem opcije **-v** mogu se ispisati:
 - Detaljne informacije o komandama: prikazuje sve interne naredbe koje GCC pokreće, kao što su cc1plus za kompajliranje C++ koda, as za assembler, i collect2 za povezivanje.
 - Pretraživački putovi: prikazuje putanje do direktorija u kojima traži zaglavlja (headers) i biblioteke (libraries).
 - Verzije i konfiguracija: prikazuje verzije GCC-a i njegove konfiguracijske opcije.
 - Podaci o bibliotekama i linkerima: detaljno prikazuje procese povezivanja, uključujući korištene biblioteke i putove do njih.

Kolekcije funkcija (libovi)

- Arhiva objektnih funkcija (biblioteka, eng. *library*, *lib*) je jedna datoteka koja u sebi sadrži više funkcija prevedenih u objektni kod na način da je svaku od njih moguće izdvojiti u izvornom obliku.
- Biblioteka se kao ulaz u **gcc** koristi na isti način kao bilo koja druga objektna datoteka.
- Prilikom povezivanja (linkanja), **gcc** iz biblioteke izdvaja potrebne funkcije i koristi ih na isti način kao i druge (direktno zadane) datoteke objektnog koda.
- Naredba **ar** služi za kreiranje i uređivanje arhiva, omogućavajući dodavanje, uklanjanje i pregled funkcija unutar biblioteka.
- Ime naredbe **ar** je skraćenica za **archiver**.

Kolekcije funkcija (libovi)

- gcc -c primjer1.c primjer2.c
- **ar rcs** libprimjer.a primjer1.o primjer2.o
- Značenje opcije **r****cs**:
 - **r** (replace): zamjena postojećih datoteka ili dodavanje novih u arhivu
 - **c** (create): kreiranje arhive ako ne postoji
 - **s** (index, **s**ymbol table): indeksiranje arhive za brže povezivanje

Rutina **make**

- Rutina **make** je alat koji automatizira proces kompajliranja i povezivanja programa prema pravilima definiranim u datoteci koja se najčešće (ukoliko nije drugačije navedeno) zove **Makefile**.
- Sintaksa: **make -f <datoteka> <opcije>**
- Primjer: **make -f moj_makefile**
- Opcija „**-f**” u gornjim naredbama znači da rutina **make** ne traži zadani "Makefile" ili "makefile", nego koristi točno navedenu datoteku **moj_makefile**.
- Glavna prednost **make** rutine je što izvršava samo one naredbe koje su potrebne, odnosno samo one dijelove koda koji su promijenjeni ili zastarjeli u odnosu na prethodne rezultate.

Struktura **Makefile** datoteke

- Sintaksa: **target: dependencies**
commands
- **target** – naziv pravila (ime programa) koje želimo pozvati
- **dependencies** – datoteke o kojima ovisi prevođenje
- **commands** – akcija koja se izvršava za generiranje datoteke
- Redak sa naredbama obavezno započinje sa <tab>.

Struktura **Makefile** datoteke

vjezba37: vjezba.c

gcc vjezba.c -o vjezba37

Struktura **Makefile** datoteke

Ovo želimo dobiti.

vjezba37: vjezba.c

gcc vjezba.c -o vjezba37

Struktura **Makefile** datoteke

Ovo želimo dobiti.

vjezba37: vjezba.c Ovo nam treba da to dobijemo.

gcc vjezba.c -o vjezba37

Struktura **Makefile** datoteke

Ovo želimo dobiti.

vjezba37: vjezba.c

Ovo nam treba da to dobijemo.

gcc vjezba.c -o vjezba37

Ovo je naredba pomoću koje ćemo to ostvariti.

Struktura **Makefile** datoteke

program: main.o funkcije.o

gcc main.o funkcije.o -o program

main.o: main.c

gcc -c main.c

funkcije.o: funkcije.c

gcc -c funkcije.c

Struktura **Makefile** datoteke

```
program: main.o funkcije.o
```

```
gcc main.o funkcije.o -o program
```

```
main.o: main.c
```

```
gcc -c main.c
```

```
funkcije.o: funkcije.c
```

```
gcc -c funkcije.c
```

Što mislite zašto ovdje nismo
koristili main.c i funkcije.c?

Literatura

- Slika na slajdu „Sadržaj” preuzeta iz:
<https://pixabay.com/illustrations/office-laptop-work-smartphone-9582092/>
- [1] By Patafisik - Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=147253964>
- [2] „Philosophy of the GNU Project”,
<https://www.gnu.org/philosophy/philosophy.en.html>